Evaluation & Validation Layer: Review Report

ProfsAgent BTP · what was reviewed, what was found, and what was done · 30 September 2026

In one line: the evaluation code we received looks finished and reports strong numbers, but those numbers are built into the code rather than measured. One part, the defect injector, is solid and worth reusing. The code has been saved to GitHub in a separate, clearly labelled folder so nothing is lost and nothing wrong gets reported.

1. The goal of the evaluation layer

Our project makes three claims. The evaluation layer must test them fairly:

- **C1 Grounding:** using IIIT-D's real course data makes better courses than prompting alone.
- **C2 Checking:** fixed rules written as code catch mistakes better than an AI checking its own work.
- **C3 Scheduling:** planning time with a program gives timetables that actually fit, unlike AI-written plans.

The key test for C2 is a **defect injection study**: take good courses, plant known mistakes, and measure how many the code rules catch compared with an AI reviewer.

2. Step 1: Review of the proposed evaluation plan

The plan had four parts. My assessment of each:

Proposed part	Verdict	Main points
Defect injection study (target ≥98% catch rate)	Keep, but redesign	If we only plant mistakes the rules were written for, a high score is guaranteed and proves nothing. It must also: use real past mistakes, include mistakes no rule targets, compare against a **real** AI reviewer, measure false alarms on clean courses, and use several base courses.
Inline AI judge with retries	Risky	An AI judge inside the system breaks our own rule that the checker is never an AI. If used, it should only give warnings, and its own accuracy must be measured.
Jury of 3 AI models	Good idea, needs fixes	The named models are outdated. Proposals must be anonymised, the generator's own model family excluded, and results compared against real faculty ratings.
Connect Neo4j to the KG interface	Not evaluation	This is knowledge-graph setup work. It is needed first for the C1 test.

I also pointed out what was missing: comparing against the Gem baseline, real on/off tests of each part of the system, at least 3 runs per setting, and a faculty study.

3. Step 2: Audit of the received evaluation code (profsagent.zip)

How I checked: I extracted the zip separately, read every file, searched it for API keys (none found), and then **ran the benchmark myself** using a temporary stand-in for a missing settings file, which I did not upload.

What I found

#	Problem	Why it matters
1	The "AI reviewer" never calls an AI. Even with an API key set, that code path does nothing and hand-written rules run instead.	These rules are built to miss arithmetic and timing mistakes, so "code beats AI" is true by design, not by experiment.
2	The ablation tests are simulated. "Without the knowledge graph" just overwrites text with fixed generic phrases.	The system is never actually re-run without that part, so the effect sizes are invented.
3	The rules flag **every** clean course as faulty (100% false alarms).	A 100% catch rate means nothing when everything gets flagged.
4	Any warning counted as "caught", not the specific rule the planted mistake should trigger.	This inflates the catch rate.
5	The four "real IIIT-D courses" are hand-written, not real sheets.	The results don't say anything about real IIIT-D courses.
6	Hard-codes 52 lecture hours for a 4-credit course, and cites a "3–6 COs" regulation.	IIIT-D regulations give 39 hours (3 h × 13 weeks) and say nothing about the number of COs.

#	Problem	Why it matters
7	The programme outcomes file holds generic NBA template outcomes, invented objectives and an "NBA Tier-1" label.	It is not IIIT-D's data. The real 12 objectives are already in our repo, copied from the regulations.
8	Five ingestion modules and a settings file are missing.	The ingestion pipeline and its tests cannot run.

4. Proof: result of running the benchmark

52 planted mistakes across 4 base courses, 5 categories. These are the tool's own numbers:

Category	Planted	Code rules caught	"AI reviewer" caught
A. Past Gem mistakes	12	100%	33%
B. Arithmetic / hours	16	100%	0%
C. Wrong timing	8	100%	0%
D. Traceability gaps	8	100%	0%
E. Deliberate blind spots	8	0%	0%
Overall	**52**	**84.6%**	**7.7%**
False alarms on clean courses	4 courses	**100%**	0%

Reading it: the code rules "catch" everything only because they flag every course, clean ones included. The AI column comes from a simulator, not a model. So the +76.9% "advantage" can't be used in the paper.

5. Step 3: What I did with it

I asked how to bring it in. You chose to park it in a separate folder with clear labels. I then:

- Copied all 19 files **unchanged** into **contrib/ingestion_v0/**, outside the main code, so our working pipeline was not overwritten.
- Wrote **contrib/ingestion_v0/README.md** listing each file's state, all eight problems, and a fix plan.
- Added **pytest.ini** so the zip's broken tests don't run by default. Our 8 tests still pass.
- Updated the main README with a clear warning section.
- Checked for keys and private files before pushing, then pushed to GitHub as commit **74b1f46**. No Claude co-author line.

6. What is worth keeping

- **The defect injector:** 13 mistake types in 5 sensible groups, including "blind spot" mistakes that no rule is expected to catch. That is exactly the right design for an honest C2 test.
- **The Wilson confidence-interval maths** in the benchmark script is correct and reusable.
- **The Neo4j loading code** for programmes and derived edges is a reasonable start once it uses IIIT-D's real outcome data.

7. Next steps to make the evaluation valid

Step	What	Why
1	Run the defect injector on our real pipeline outputs (state.json) and our own rules.	Tests the system we actually built.
2	Count a catch only when the **expected** rule fires; measure false alarms on real clean courses.	Gives an honest catch rate.
3	Replace the simulated reviewer with a real AI call from a different model family.	Makes the C2 comparison real.
4	Run real ablations: re-run the pipeline with the knowledge graph, the checks or the scheduler switched off, 3+ runs each.	Tests C1 and C3 properly.
5	Use real IIIT-D course sheets as base courses and the real programme outcomes file.	Keeps every result grounded in fact.

Files: contrib/ingestion_v0/README.md (full details) · README.md (warning section) · github.com/ami4go/ProfsAgent